



UNIVERSIDAD NACIONAL AUTONOMA DE
MÉXICO

FACULTAD DE INGENIERIA

DIVISIÓN DE INGENIERIA ELECTRICA

INGENIERIA EN COMPUTACIÓN

COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA



LAB S07: Mexi Banco: directorio, gateway y balanceo

Nombre de el alumno: Guerrero López Uriel Iván

Nº de Cuenta: 320046045

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 10/03/2026

CALIFICACIÓN: _____

PASO 1: La historia

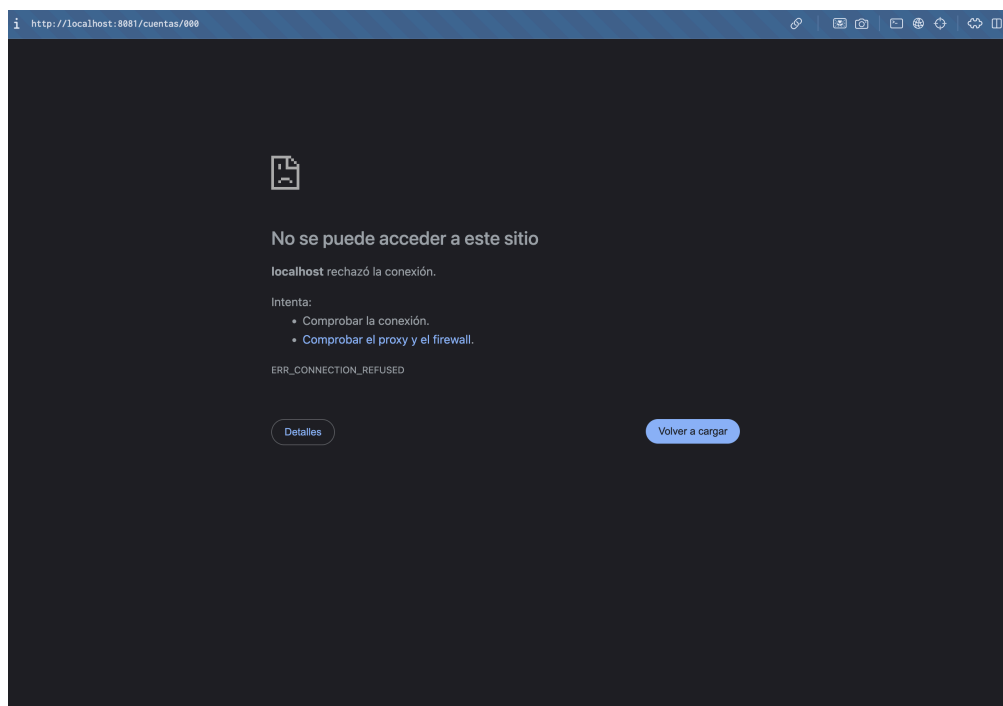
Al separar la aplicación en microservicios, las llamadas directas entre métodos dejaron de ser suficientes, porque cada componente pasó a ejecutarse de forma independiente. La comunicación tuvo que realizarse mediante HTTP, considerando direcciones, puertos, respuestas y posibles errores de conexión. Esto también generó el problema de mantener las direcciones actualizadas cuando los servicios cambian o tienen varias copias.

```
UP192@MacBook-Air-de-Ufio1 mexi-banco % docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
mexi-banco-v86-cuenta-1	sha256:2f5f21f2f6120eda1836e0d53daa38a631617925cb817213504d2d9abc85346	"java -jar app.jar"	cuenta	6 days ago	Up 15 seconds (healthy)	0.0.0.0:8081->8080/t
cp, [::]:8081->8080/tcp						
mexi-banco-v86-db-1	postgres:17-alpine	"docker-entrypoint.s."	db	6 days ago	Up 20 seconds (healthy)	5432/tcp
mexi-banco-v86-movimiento-1	sha256:dd96c71805e8b75318346f357667b4abdf1dd799e59828139e12696195574aaf	"java -jar app.jar"	movimiento	6 days ago	Up 15 seconds (healthy)	0.0.0.0:8082->8080/t
cp, [::]:8082->8080/tcp						
mexi-banco-v86-notificacion-1	sha256:42aed7e4588421f908b9238d208c955a72d227cfb8534e76113b46fce8d791a2	"java -jar app.jar"	notificacion	6 days ago	Up 15 seconds (healthy)	0.0.0.0:8084->8080/t
cp, [::]:8084->8080/tcp						
mexi-banco-v86-spei-1	sha256:3bd8511ac7f34bfcc371ef7c5a8e6ec490b5f1b1448aac9775fd49ca6a45ab3a	"java -jar app.jar"	spei	6 days ago	Up 15 seconds (healthy)	0.0.0.0:8085->8080/t
cp, [::]:8085->8080/tcp						
mexi-banco-v86-transferencia-1	sha256:7ad54929d778b9232d53d8ba3f1ba73f02f8f54b1337b7eadaaae6fc9f94933c	"java -jar app.jar"	transferencia	6 days ago	Up 15 seconds (healthy)	0.0.0.0:8083->8080/t
cp, [::]:8083->8080/tcp						

PASO 2: Directorio

El directorio Eureka permite registrar los servicios y consultar dónde se encuentran disponibles. Al inicio, el tablero no cargaba porque el directorio no estaba disponible. Después de levantar los servicios, fue posible consultar el tablero y observar las instancias registradas.



Después: Se agregó la anotación `@EnableEurekaServer` en la clase `DiscoveryApplication.java` para habilitar Eureka como servidor de descubrimiento. También se configuró para que no intentara registrarse a sí mismo ni consultar un directorio externo, debido a que en esta práctica se utiliza una sola instancia de Eureka.

Después: Se reconstruyeron y levantaron los contenedores con `docker compose up -d --build`. Al ingresar a `http://localhost:8761`, se mostró el panel de Eureka con los cinco microservicios CUENTA, MOVIMIENTO, NOTIFICACION, SPEI y TRANSFERENCIA registrados en estado UP (1). Esto confirmó que los servicios podían anunciar su disponibilidad en el directorio de manera automática.

The screenshot shows the Spring Eureka dashboard at `http://localhost:8761`. The dashboard includes a header with the Spring Eureka logo and navigation links. The main content area is divided into several sections:

- System Status:** A table showing environment details.

Environment	test	Current time	2026-10-10T11:42:31 +0000
Data center	default	Uptime	00:00
		Lease expiration enabled	true
		Renews threshold	10
		Renews (last min)	0
- THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.**
- DS Replicas:** A section showing the local host `localhost` as a data center replica.
- Instances currently registered with Eureka:** A table listing the registered services.

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 92c3f3e10546:cuenta:8080
DISCOVERY	n/a (1)	(1)	UP (1) - 89f342e5342d:discovery:8761
MOVIMIENTO	n/a (1)	(1)	UP (1) - 3aa5d68bd202:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - e7cd5c513845:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 0edfa212b82f:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - efa8c878b9f7:transferencia:8080
- General Info:** A table showing system metrics.

Name	Value
total-avail-memory	94mb
num-of-cpus	10
current-memory-usage	74mb (78%)

Después: Se consultó `http://localhost:8761/eureka/apps/CUENTA` para visualizar la información registrada del servicio CUENTA en formato XML. La respuesta permitió identificar datos como el nombre del servicio, el host, el puerto y el estado UP, además de

los intervalos configurados para renovar y expirar el registro. Con ello se comprobó que Eureka también permite consultar la información de los servicios mediante una API.

```
<?xml version='1.0' encoding='UTF-8'>
<application>
  <name>Cuenta</name>
  <instance>
    <instanceId>b5447370ae3f:cuenta:8080</instanceId>
    <hostName>172.22.0.9</hostName>
    <app>Cuenta</app>
    <ipAddr>172.22.0.9</ipAddr>
    <status>UP</status>
    <overriddenStatus>UNKNOWN</overriddenStatus>
    <port enabled='true'>8080</port>
    <securePort enabled='false'>443</securePort>
    <countryId>1</countryId>
    <dataCenterInfo class='com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo'>
      <name>MyOwn</name>
    </dataCenterInfo>
    <leaseInfo>
      <renewalIntervalInSecs>5</renewalIntervalInSecs>
      <durationInSecs>15</durationInSecs>
      <registrationTimestamp>1791634575017</registrationTimestamp>
      <lastRenewalTimestamp>1791634580271</lastRenewalTimestamp>
      <evictionTimestamp>0</evictionTimestamp>
      <serviceUpTimestamp>1791634552859</serviceUpTimestamp>
    </leaseInfo>
    <metadata>
      <management.port>8080</management.port>
    </metadata>
    <homePageUrl>http://172.22.0.9:8080</homePageUrl>
    <statusPageUrl>http://172.22.0.9:8080/actuator/info</statusPageUrl>
    <healthCheckUrl>http://172.22.0.9:8080/actuator/health</healthCheckUrl>
    <vipAddress>cuenta</vipAddress>
    <secureVipAddress>cuenta</secureVipAddress>
    <isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
    <lastUpdatedTimestamp>1791634575017</lastUpdatedTimestamp>
    <lastDirtyTimestamp>1791634552797</lastDirtyTimestamp>
    <actionType>ADDED</actionType>
  </instance>
  <instance>
    <instanceId>542436dd2cc9:cuenta:8080</instanceId>
    <hostName>172.22.0.10</hostName>
    <app>Cuenta</app>
    <ipAddr>172.22.0.10</ipAddr>
    <status>UP</status>
    <overriddenStatus>UNKNOWN</overriddenStatus>
    <port enabled='true'>8080</port>
    <securePort enabled='false'>443</securePort>
    <countryId>1</countryId>
    <dataCenterInfo class='com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo'>
      <name>MyOwn</name>
    </dataCenterInfo>
    <leaseInfo>
      <renewalIntervalInSecs>5</renewalIntervalInSecs>
      <durationInSecs>15</durationInSecs>
      <registrationTimestamp>1791634575017</registrationTimestamp>
      <lastRenewalTimestamp>1791634584167</lastRenewalTimestamp>
      <evictionTimestamp>0</evictionTimestamp>
      <serviceUpTimestamp>1791634552303</serviceUpTimestamp>
    </leaseInfo>
    <metadata>
      <management.port>8080</management.port>
    </metadata>
    <homePageUrl>http://172.22.0.10:8080</homePageUrl>
    <statusPageUrl>http://172.22.0.10:8080/actuator/info</statusPageUrl>
    <healthCheckUrl>http://172.22.0.10:8080/actuator/health</healthCheckUrl>
    <vipAddress>cuenta</vipAddress>
    <secureVipAddress>cuenta</secureVipAddress>
    <isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
    <lastUpdatedTimestamp>1791634575017</lastUpdatedTimestamp>
    <lastDirtyTimestamp>1791634552218</lastDirtyTimestamp>
    <actionType>ADDED</actionType>
  </instance>
  <instance>
    <instanceId>brfa1e4b55fd:cuenta:8080</instanceId>
    <hostName>172.22.0.5</hostName>
    <app>Cuenta</app>
    <ipAddr>172.22.0.5</ipAddr>
    <status>UP</status>
    <overriddenStatus>UNKNOWN</overriddenStatus>
    <port enabled='true'>8080</port>
    <securePort enabled='false'>443</securePort>
    <countryId>1</countryId>
    <dataCenterInfo class='com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo'>
      <name>MyOwn</name>
    </dataCenterInfo>
    <leaseInfo>
      <renewalIntervalInSecs>5</renewalIntervalInSecs>
      <durationInSecs>15</durationInSecs>
      <registrationTimestamp>1791634575016</registrationTimestamp>
      <lastRenewalTimestamp>1791634580271</lastRenewalTimestamp>
      <evictionTimestamp>0</evictionTimestamp>
      <serviceUpTimestamp>1791634553282</serviceUpTimestamp>
    </leaseInfo>
    <metadata>
      <management.port>8080</management.port>
    </metadata>
    <homePageUrl>http://172.22.0.5:8080</homePageUrl>
    <statusPageUrl>http://172.22.0.5:8080/actuator/info</statusPageUrl>
    <healthCheckUrl>http://172.22.0.5:8080/actuator/health</healthCheckUrl>
    <vipAddress>cuenta</vipAddress>
    <secureVipAddress>cuenta</secureVipAddress>
    <isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
    <lastUpdatedTimestamp>1791634575016</lastUpdatedTimestamp>
    <lastDirtyTimestamp>1791634553227</lastDirtyTimestamp>
    <actionType>ADDED</actionType>
  </instance>
</application>
```

Pregunta 1. ¿Por qué NOTIFICACION no desaparece al instante del directorio?

NOTIFICACION no desaparece inmediatamente porque Eureka espera el plazo de expiración configurado de 15 segundos para considerar que una instancia dejó de responder. Durante ese tiempo, el servicio puede seguir apareciendo en el directorio aunque

el contenedor esté detenido. Por eso, en el ensayo tardó aproximadamente 18 segundos en desaparecer.

Pregunta 2. ¿TRANSFERENCIA está usando el directorio Eureka para encontrar a CUENTA?

No. Aunque CUENTA ya está registrado en Eureka, TRANSFERENCIA todavía utiliza la dirección `http://cuenta:8080`, definida manualmente en el archivo `docker-compose.yml`. Esto significa que todavía no está aprovechando el descubrimiento de servicios de Eureka para localizar a CUENTA. Para hacerlo, será necesario configurar la comunicación mediante el nombre lógico del servicio y habilitar el mecanismo de descubrimiento y balanceo de carga.

PASO 3: Gateway

Antes: Se ejecutó `curl -i -X POST http://localhost:8082/movimientos` para comprobar el acceso directo al servicio MOVIMIENTO desde la computadora. La petición devolvió un código HTTP 400, lo que indica que el endpoint existe, pero faltan datos obligatorios para procesar la solicitud. Esto demuestra que la operación podía alcanzarse directamente sin pasar por un gateway.

```
ur182@MacBook-Air-de-Uriel mexi-banco % curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sat, 10 Oct 2026 11:44:34 GMT
Connection: close
```

Se consultó la CLABE inexistente 000 mediante el API Gateway, utilizando el puerto 8080. La petición fue dirigida correctamente al microservicio CUENTA, que respondió con HTTP 404 al indicar que la cuenta solicitada no existía. La cabecera X-Instancia permite identificar que la respuesta provino de una instancia del microservicio.

```
ur182@MacBook-Air-de-Uriel mexi-banco % curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 92c3f3e10546
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 11:52:30 GMT
```

Se intentó registrar un movimiento mediante el API Gateway con una petición POST a /api/movimientos. El Gateway respondió con HTTP 404 porque la ruta configurada para MOVIMIENTO acepta únicamente solicitudes GET. Por lo tanto, la petición fue rechazada en la entrada y no llegó al microservicio.

```
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}%
-X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-10T11:52:43.422+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"f6150530-12"}%
BE="S07$(date +%s)"
```

Se creó una cuenta mediante una petición POST al endpoint /api/cuentas, utilizando el puerto 8080 del Gateway y una CLABE única. El servidor respondió con HTTP 201 Created y devolvió los datos de la cuenta en formato JSON, con un saldo inicial de cero. Esto confirmó que el Gateway podía dirigir correctamente las solicitudes de creación hacia CUENTA.

```
URL="http://localhost:8080/api/cuentas/$LAB_CLABE"
curl -i "$URL"
echo "$URL"
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 92c3f3e10546
Content-Type: application/json
Date: Sat, 10 Oct 2026 11:53:00 GMT

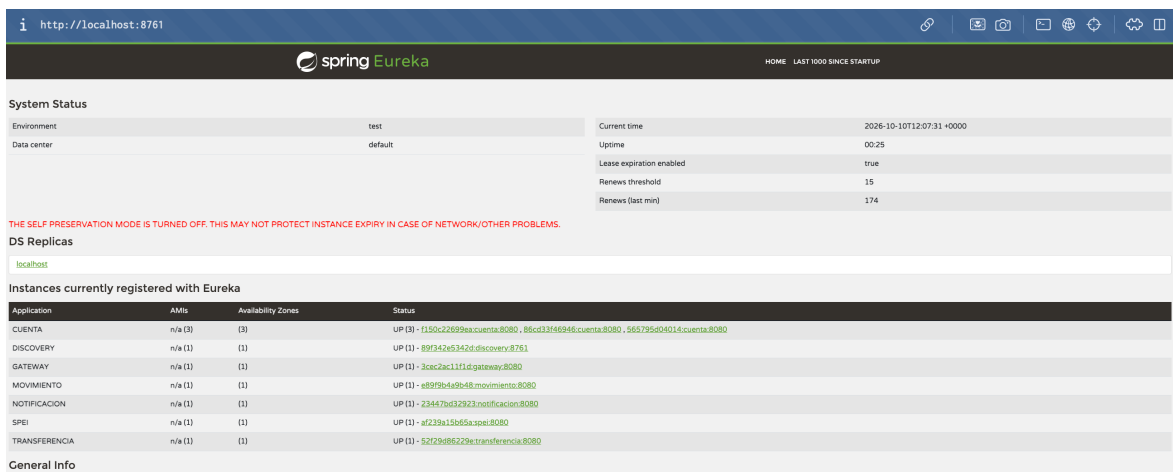
{"id":1,"clabe":"S071791633180","titular":"S07","saldo":0}HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 92c3f3e10546
Content-Type: application/json
Date: Sat, 10 Oct 2026 11:53:00 GMT
```

Se consultó la cuenta recién creada mediante su CLABE, utilizando el endpoint /api/cuentas/{CLABE} a través del puerto 8080. La respuesta HTTP 200 OK mostró la información de la cuenta en formato JSON, confirmando que la operación de consulta funcionaba correctamente mediante el Gateway.

Paso 4. Réplicas y balanceo de carga

Se modificó el archivo docker-compose.yml para configurar tres réplicas del microservicio CUENTA mediante deploy: replicas: 3. También se eliminó la publicación directa de sus puertos hacia la computadora, de manera que las instancias se administraran dentro de Docker y se accediera a ellas a través del Gateway.

Después de levantar los contenedores, se consultó el panel de Eureka en <http://localhost:8761>. El servicio CUENTA apareció con tres instancias en estado UP (3), confirmando que las réplicas se habían registrado correctamente en el directorio.



The screenshot shows the Spring Eureka dashboard at <http://localhost:8761>. The page title is "spring Eureka". The "System Status" section shows the environment as "test" and the data center as "default". The current time is "2026-10-10T12:07:31 +0000" and the uptime is "00:29". The "DS Replicas" section shows "localhost". The "Instances currently registered with Eureka" section shows a table with columns: Application, AMIs, Availability Zones, and Status. The table lists several applications: CUENTA, DISCOVERY, GATEWAY, MOVIMIENTO, NOTIFICACION, SPEI, and TRANSFERENCIA. Each application has three instances in the "UP" state. The "General Info" section is also visible.

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - f150c22699ea.cuenta.8080, 86cd33f46946.cuenta.8080, 565795d04014.cuenta.8080
DISCOVERY	n/a (1)	(1)	UP (1) - 89f342e5342d.discovery.8761
GATEWAY	n/a (1)	(1)	UP (1) - 3ccc2ac131fd.gateway.8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - e89f9bda9e48.movimiento.8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 23447b632923.notificacion.8080
SPEI	n/a (1)	(1)	UP (1) - af229a15b65a.spei.8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 52f29d865229.transferencia.8080

Se realizaron nueve consultas consecutivas a través del Gateway y se revisó la cabecera X-Instancia de cada respuesta. Se observó una alternancia entre cuenta-1, cuenta-2 y cuenta-3, lo que demuestra que las solicitudes se distribuyeron secuencialmente entre las tres instancias mediante el algoritmo Round Robin.

```
ur182@MacBook-Air-de-Uriel mexi-banco % for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: 565795d04014
X-Instancia: f150c22699ea
X-Instancia: 86cd33f46946
X-Instancia: 565795d04014
X-Instancia: f150c22699ea
X-Instancia: 86cd33f46946
X-Instancia: 565795d04014
X-Instancia: f150c22699ea
X-Instancia: 86cd33f46946
```

Paso 5. Simulación de caída y recuperación

Antes de detener una instancia, se predijo que las solicitudes dirigidas a la réplica caída podrían producir errores HTTP 500 o 503, mientras que las demás instancias continuarían atendiendo peticiones. Se esperaba que los errores desaparecieran cuando Eureka eliminara la instancia caída y el Gateway actualizara su lista de destinos.

Se ejecutó `docker kill mexi-banco-v06-cuenta-1` para detener abruptamente una instancia de CUENTA. La prueba permitió simular una falla inesperada y observar cómo reaccionaban Eureka y el Gateway ante la pérdida de una de las réplicas.

```
ur182@MacBook-Air-de-Uriel mexi-banco % docker compose logs -f cuenta | grep atendida
cuenta-2 | 2026-10-10T12:03:19.377Z INFO 1 --- [cuenta] [nio-8888-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791633180 atendida por f150c22699ea
cuenta-2 | 2026-10-10T12:03:19.450Z INFO 1 --- [cuenta] [nio-8888-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791633180 atendida por f150c22699ea
cuenta-2 | 2026-10-10T12:03:19.492Z INFO 1 --- [cuenta] [nio-8888-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791633180 atendida por f150c22699ea
cuenta-1 | 2026-10-10T12:03:19.503Z INFO 1 --- [cuenta] [nio-8888-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791633180 atendida por 86cd33f46946
cuenta-1 | 2026-10-10T12:03:19.664Z INFO 1 --- [cuenta] [nio-8888-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791633180 atendida por 86cd33f46946
cuenta-1 | 2026-10-10T12:03:19.708Z INFO 1 --- [cuenta] [nio-8888-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791633180 atendida por 86cd33f46946
cuenta-3 | 2026-10-10T12:03:19.748Z INFO 1 --- [cuenta] [nio-8888-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791633180 atendida por 565795d04014
cuenta-3 | 2026-10-10T12:03:19.627Z INFO 1 --- [cuenta] [nio-8888-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791633180 atendida por 565795d04014
cuenta-3 | 2026-10-10T12:03:19.678Z INFO 1 --- [cuenta] [nio-8888-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791633180 atendida por 565795d04014
^C
```

```
ur182@MacBook-Air-de-Uriel mexi-banco % date +%T
06:06:42
```

```
ur182@MacBook-Air-de-Uriel mexi-banco % docker kill mexi-banco-v06-cuenta-1
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 86cd33f4694655148512e6413c622d5b9a7d5da7f5a8a86dd2fc1798d53c5145 is not running
ur182@MacBook-Air-de-Uriel mexi-banco % docker start mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
```

- Hora del apagado de la instancia: aproximadamente 06:06, tomando como referencia la hora consultada en la terminal. No se pudo confirmar el segundo exacto del apagado.
- Hora de salida del directorio: Aproximadamente 29 segundos después.
- Cantidad de errores observados: 8 errores en un total de 45 peticiones.

¿Por qué ocurre esto? Porque el Gateway mantiene una caché con las instancias registradas en Eureka y no actualiza esa información de manera instantánea. Después de apagar una réplica, puede continuar enviándole solicitudes durante algunos segundos, lo que provoca errores. Cuando Eureka detecta la caída y el Gateway actualiza su lista de instancias disponibles, las peticiones se dirigen a las réplicas que siguen funcionando.

En conclusión, durante esta práctica aprendí cómo se comunican los microservicios y cómo se puede mejorar su funcionamiento utilizando Eureka, Gateway y el balanceador de carga.

Pieza	Qué problema resuelve	Dónde lo comprobé
Directorio	Permite localizar los servicios por su nombre sin configurar manualmente sus direcciones.	En el tablero de Eureka, en localhost:8761.
Gateway	Permite tener una sola entrada al sistema y controlar qué peticiones pueden pasar a cada servicio.	En las pruebas con respuestas HTTP 400 y 404.
Balanceador	Distribuye las peticiones entre varias copias de un servicio.	En las consultas donde cambió el valor de X-Instancia.

El directorio resuelve la localización de los servicios. El Gateway resuelve el acceso centralizado y el control de las peticiones. El balanceador resuelve la distribución del tráfico entre varias réplicas. Cuando murió una copia, observé errores HTTP 500 y 503 porque Eureka y el Gateway necesitan tiempo para detectar la caída y actualizar las instancias disponibles.

Tarea

```
=== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009163456101","titular":"Ana","saldoInicial":1000}'
{
  "id": 2,
  "clave": "002180009163456101",
  "titular": "Ana",
  "saldo": 1000
}
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

=== 2. Abrir la cuenta de Beto con 500 (misma puerta) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009163456102","titular":"Beto","saldoInicial":500}'
{
  "id": 3,
  "clave": "002180009163456102",
  "titular": "Beto",
  "saldo": 500
}
[HTTP 201]

=== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009163456101","claveDestino":"002180009163456102","monto":200}'
{
  "id": 1,
  "claveOrigen": "002180009163456101",
  "claveDestino": "002180009163456102",
  "monto": 200,
  "fecha": "2026-10-10T12:16:03.352892799Z"
}
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-2 GET /cuentas/002180009163456101 atendida por b5447370ae3f
cuenta-1 GET /cuentas/002180009163456102 atendida por b7ba1e4b59fd
cuenta-3 POST /cuentas/002180009163456101/cargos atendida por 542436dd2cc9
cuenta-2 POST /cuentas/002180009163456102/abonos atendida por b5447370ae3f
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

=== 4. Consultar esa transferencia ===
$ curl http://localhost:8080/api/transferencias/1
{
  "id": 1,
  "claveOrigen": "002180009163456101",
  "claveDestino": "002180009163456102",
  "monto": 200.00,
  "fecha": "2026-10-10T12:16:03.352893Z"
}
[HTTP 200]

=== 5. Saldo: el dinero no se crea ni se pierde ===
$ curl http://localhost:8080/api/cuentas/002180009163456101
{
  "id": 2,
  "clave": "002180009163456101",
  "titular": "Ana",
  "saldo": 800.00
}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009163456102
```

```

=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===
$ curl http://localhost:8080/api/movimientos?clave=002180009163456101
{
  "id": 3,
  "claveCuenta": "002180009163456101",
  "tipo": "TRANSFERENCIA_ENVIADA",
  "monto": -200.00,
  "saldoResultante": 800.00,
  "fecha": "2026-10-10T12:16:03.325116Z",
  "referencia": "a 002180009163456102"
},
{
  "id": 1,
  "claveCuenta": "002180009163456101",
  "tipo": "DEPOSITO",
  "monto": 1000.00,
  "saldoResultante": 1000.00,
  "fecha": "2026-10-10T12:16:01.737372Z",
  "referencia": "saldo inicial"
}
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clave=002180009163456102
{
  "id": 2,
  "claveCuenta": "002180009163456102",
  "mensaje": "TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00",
  "fecha": "2026-10-10T12:16:03.586670Z"
}
[HTTP 200]

=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91634561' -d '{"claveOrigen":"002180009163456101","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{
  "id": 1,
  "idempotencyKey": "spei-demo-91634561",
  "claveOrigen": "002180009163456101",
  "bancoDestino": "BANCO-X",
  "claveDestino": "0121800000000000009",
  "monto": 50,
  "estado": "ENVIADO"
}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91634561' -d '{"claveOrigen":"002180009163456101","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{
  "id": 1,
  "idempotencyKey": "spei-demo-91634561",
  "claveOrigen": "002180009163456101",
  "bancoDestino": "BANCO-X",
  "claveDestino": "0121800000000000009",
  "monto": 50.00,
  "estado": "ENVIADO"
}
[HTTP 201]

```

```

=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ===
consulta 1 X-Instancia: b5447370ae3f (mexi-banco-v06-cuenta-2)
consulta 2 X-Instancia: bfba1e4b55fd (mexi-banco-v06-cuenta-1)
consulta 3 X-Instancia: 542436dd2cc9 (mexi-banco-v06-cuenta-3)
consulta 4 X-Instancia: b5447370ae3f (mexi-banco-v06-cuenta-2)
consulta 5 X-Instancia: bfba1e4b55fd (mexi-banco-v06-cuenta-1)
consulta 6 X-Instancia: 542436dd2cc9 (mexi-banco-v06-cuenta-3)
consulta 7 X-Instancia: b5447370ae3f (mexi-banco-v06-cuenta-2)
consulta 8 X-Instancia: bfba1e4b55fd (mexi-banco-v06-cuenta-1)
consulta 9 X-Instancia: 542436dd2cc9 (mexi-banco-v06-cuenta-3)
-> 3 replicas distintas en 9 consultas; 6 veces se repitió la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

=== 9. Los endpoints internos no tienen puerta ===
$ curl -X POST http://localhost:8080/api/cuentas/002180009163456101/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{
  "timestamp": "2026-10-10T12:16:06.224400000",
  "path": "/api/cuentas/002180009163456101/abonos",
  "status": 404,
  "error": "Not Found",
  "requestId": "071ad5fa-26"
}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009163456101","claveDestino":"002180009163456199","monto":10}'
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009163456199",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009163456101","claveDestino":"002180009163456102","monto":99999}'
{
  "type": "about:blank",
  "title": "Unprocessable Entity",
  "status": 422,
  "detail": "Saldo insuficiente en la CLABE 002180009163456101",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 422]

=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido
uri82@MacBook-Air-de-Uriel mexi-banco % docker compose logs cuenta | grep -E 'cargos|abonos'
cuenta-2 | 2026-10-10T12:16:03.297Z INFO 1 --- [cuenta] [nio-8080-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009163456102/abonos atendida por b5447370ae3f
cuenta-1 | 2026-10-10T12:16:06.244Z INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009163456101/cargos atendida por bfba1e4b55fd
cuenta-3 | 2026-10-10T12:16:03.273Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009163456101/cargos atendida por 542436dd2cc9
cuenta-3 | 2026-10-10T12:16:06.308Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009163456101/cargos atendida por 542436dd2cc9
uri82@MacBook-Air-de-Uriel mexi-banco %

```

Con esta tarea aprendí a configurar el cliente HTTP de Transferencia para que utilice el descubrimiento de servicios de Eureka y el balanceo de carga mediante la anotación `@LoadBalanced`. Antes, Transferencia se conectaba a CUENTA mediante una URL fija y dependía del DNS de Docker; ahora puede consultar las instancias registradas y distribuir las peticiones entre las tres réplicas. También comprendí que el cargo y el abono de una

misma transferencia pueden ser atendidos por instancias diferentes. Finalmente, las pruebas permiten comprobar los saldos, la idempotencia del SPEI y que los errores controlados no modifiquen el dinero de las cuentas.